

Why Braking? Scenario Extraction and Reasoning Utilizing LLM

Yin Wu^{1,2}, Daniel Slieter¹, Vivek Subramanian¹,
Ahmed Abouelazm³, Robin Bohn¹, and J. Marius Zöllner^{2,3}

Abstract—The growing number of ADAS-equipped vehicles has led to a dramatic increase in driving data, yet most of them capture routine driving behavior. Identifying and understanding safety-critical corner cases within this vast dataset remains a significant challenge. Braking events are particularly indicative of potentially hazardous situations, motivating the central question of our research: *Why does a vehicle brake?* Existing approaches primarily rely on rule-based heuristics to retrieve target scenarios using predefined condition filters. While effective in simple environments such as highways, these methods lack generalization in complex urban settings. In this paper, we propose a novel framework that leverages Large Language Model (LLM) for scenario understanding and reasoning. Our method bridges the gap between low-level numerical signals and natural language descriptions, enabling LLM to interpret and classify driving scenarios. We propose a dual-path scenario retrieval that supports both category-based search for known scenarios and embedding-based retrieval for unknown Out-of-Distribution (OOD) scenarios. To facilitate evaluation, we curate scenario annotations on the Argoverse 2 Sensor Dataset. Experimental results show that our method outperforms rule-based baselines and generalizes well to OOD scenarios.

Index Terms—Autonomous Driving, ADAS, Scenario Extraction, LLM

I. INTRODUCTION

With the growing deployment of Advanced Driver-Assistance Systems (ADAS)-equipped vehicles, the volume of driving data collected has increased dramatically. According to [1], an autonomous vehicle produces about 4000 GB of data a day, depending on its sensor suite, such as cameras, LiDAR, radar, and BUS-Systems. While most of the data captures routine driving behaviors, only a small subset of data is relevant for functional validation. Among various driving behaviors, braking is a particularly salient signal that often correlates with potentially hazardous situations, such as obstacle avoidance, cut-ins, or yielding at intersections. Understanding why a vehicle brakes can thus reveal meaningful corner cases and enhance the coverage of scenario-based assessments. Fig. 1 illustrates four representative examples in which the ego vehicle brakes: a yellow oncoming vehicle during a left turn, a green pedestrian crossing during a right turn, a red vehicle cutting in, and a bicycle crossing while the ego proceeds straight. The ego vehicle's ADAS system must promptly brake in these four scenarios to avoid conflicts.

While a braking signal is easy to detect, identifying its cause is not. This motivates the central question of this

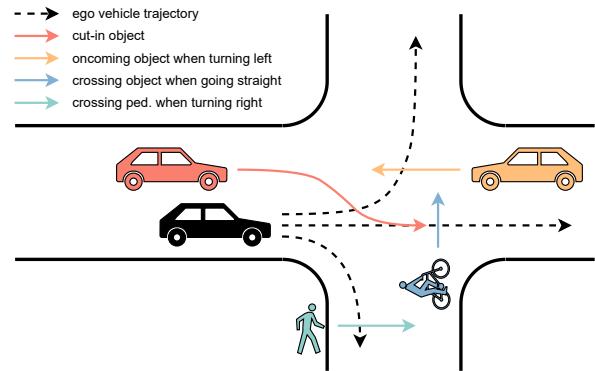


Fig. 1: Four representative scenarios where the black ego vehicle brakes to avoid obstacles: waiting for a yellow oncoming vehicle when turning left, yielding to a green pedestrian when turning right, slowing down for a red cut-in vehicle or a crossing blue bicycle when going straight.

work: *Why braking?* Our objective is to extract and reason about scenarios that causally lead to ego vehicle braking in a structured and explainable manner.

To address this challenge, various scenario extraction approaches have been explored [2]–[5]. A common paradigm is to use rule-based methods to tag agent activities [2], [3] and retrieve specific scenarios with carefully defined condition filters. However, defining these filter conditions is non-trivial: relaxing a filter may recover some false negatives (False Negative (FN)) but can also introduce new false positives (False Positive (FP)), and vice versa. Zhao et al. [4] propose using LLM to automatically generate filter conditions, yet this approach still faces similar trade-offs between recall and precision.

In this work, we move beyond using LLM solely as condition translators and instead leverage them for holistic scenario understanding. To bridge the gap between low-level numerical BUS data and high-level semantic reasoning, we first extract symbolic tags using the scenario tagging method proposed by Guo et al. [3], and convert them into structured natural language descriptions. To improve efficiency and avoid irrelevant context, we first identify the key objects that are causally related to the ego vehicle's braking event. For each identified object, we construct a scenario description that captures the interaction between the ego vehicle and the candidate. These descriptions are then fed into the LLM to perform rephrasing, contextual reasoning, and scenario

¹Authors are with the CARIAD SE, Germany
firstname.lastname@cariad.technology

²Authors are with the Karlsruhe Institute of Technology, Germany

³Authors are with the FZI Research Center for Information Technology, Germany

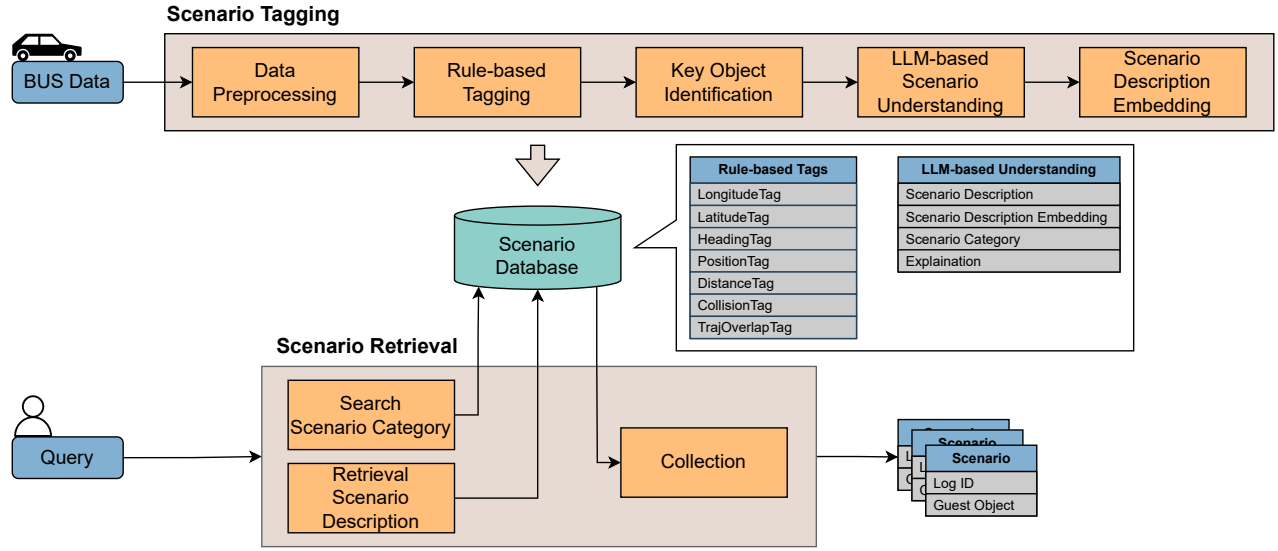


Fig. 2: The scenario tagging and retrieval framework consists of several stages. The preprocessed raw driving data is first processed using rule-based methods to generate activity tags. The module Key Object Identification (KeyIdent) then detects candidate objects that potentially trigger the ego vehicle’s braking behavior. For each ego–candidate object pair, the corresponding activity log is interpreted by an LLM to produce rephrased scenario descriptions, classify the scenario category, and give explanations. Subsequently, an embedding model generates a scenario description embedding. The enrichment information is stored in a scenario database, which is queried through a dual-path retrieval module.

classification. Subsequently, a text embedding model is used to obtain an embedding vector from the rephrased scenario descriptions. For the scenario retrieval stage, we propose a dual-path retrieval framework that supports retrieving scenarios either directly based on the LLM-generated scenario categories or via similarity search using embeddings derived from a scenario description query.

To evaluate our proposed method, we annotate 700 data logs on the training set of the Argoverse 2 Sensor Dataset [6] with scenario labels. Experimental results show that our LLM-based approach not only outperforms the rule-based baseline [3] but also demonstrates zero-shot generalization on OOD unseen scenarios.

II. RELATED WORK

In this section, we review three research areas relevant to our proposed method: scenario-based validation for ADAS, scenario extraction methodologies, and LLM-based approaches for driving scenario understanding.

A. From Mileage-Based to Scenario-Based Validation

The validation of ADAS and Automated Driving Systems (ADS) requires more than just accumulating driving distance. According to [7], demonstrating the safety of an ADS would require billions of miles of on-road testing, which is practically infeasible. Consequently, the industry has shifted towards scenario-based validation methodologies [8]. In this paradigm, an ADS is tested against a curated catalog of critical and corner-case scenarios.

A key challenge lies in identifying these crucial scenarios. While synthetic scenario generation is a popular approach [9]–[11], it may not fully capture the complexity and distribution of real-world events, potentially overlooking rare but critical corner cases. To address this, our work focuses on extracting meaningful scenarios directly from large-scale real-world driving data. Specifically, we target scenarios that cause the ego-vehicle to perform braking maneuvers, as these are directly relevant to ADAS functions like Adaptive Cruise Control (ACC), Autonomous Emergency Braking (AEB), etc.

B. Scenario Extraction from Real-World Data

Efficiently mining scenarios from vast datasets is a significant research area. An early and effective paradigm is “tagging-and-searching,” where raw data is pre-processed into a database with descriptive tags, enabling fast queries [2]. Subsequent works have focused on building the interaction tag between agents and environments to allow for more precise scenario definitions [3]. More recently, LLM have been used to translate natural language queries into formal search filters. For instance, Chat2Scenario [4] and RefAV [5] leverage LLM to find scenarios matching complex textual descriptions. However, these approaches primarily use LLM as language translators, rather than fully exploiting their capacity for contextual understanding and reasoning.

C. Large Language Models for Semantic Scenario Understanding

Modern LLM have demonstrated impressive reasoning and zero-shot capabilities across both textual and visual modali-

TABLE I: Definition of activity tags and values.

Tag Category	Category Description	Tag Name
LongitudeTag	Describes the vehicle's longitudinal motion along its path.	CRUISING STANDING_STILL ACCELERATING DECELERATING REVERSING
LatitudeTag	Describes the vehicle's orientation, including turns and lateral shifts relative to its path.	FACING_FORWARD VEERING_LEFT VEERING_RIGHT TURNING_LEFT TURNING_RIGHT
CollisionTag	Describes the collision risk between two agents.	NO LOW HIGH
HeadingTag	Describes the relative heading angle of guest to ego vehicle.	SAME LEFT RIGHT OPPOSITE
PositionTag	Describes the relative position of guest to ego vehicle.	FRONT FRONT_LEFT LEFT BACK_LEFT BACK BACK_RIGHT RIGHT FRONT_RIGHT
DistanceTag	Describes the bounding box distance between two objects.	VERY_CLOSE CLOSE MEDIUM FAR
TrajOverlapTag	Describe the overlap risk of the aggregated bounding boxes of two objects over a future time.	NO LOW HIGH

ties. Recent works such as DriveLM [12] and DriveVLM [13] integrate LLM into end-to-end autonomous driving systems, focusing primarily on single-scene understanding based on visual inputs. However, these models are not designed to directly interpret time-series driving data. For instance, a 10-second driving sequence recorded at 10 Hz contains 100 frames. Feeding all 100-frame scene descriptions into the LLM results in context overload, reduced relevance, and longer inference time.

To integrate LLM in downstream tasks, the prompting engineer plays an important role in enhancing the output quality without modifying their underlying parameters. Techniques such as chain-of-thought prompting [14] has been shown to improve reasoning capabilities and reduce output variance. Liu et al. [15] highlighted that explicitly specifying the role of the LLM, alongside enforcing structured output formats such as JSON, can significantly enhance output quality and consistency. We referred to these methods when designing our prompts.

III. METHODOLOGY

As shown in Fig. 2, our proposed scenario tagging and retrieval framework consists of two main components: a two-stage scenario tagging module and a dual-path scenario retrieval module. The scenario tagging module processes and enriches the recorded driving BUS data. The data are first

preprocessed using interpolation and smoothing algorithms. Then, a rule-based tagging module is applied to extract activity tags for each agent as well as interaction tags between the ego vehicle and surrounding agents. Subsequently, the KeyIdent module filters out non-relevant objects to reduce redundancy in the following steps. Based on these tags, the numerical data are translated into structured natural language descriptions and fed into a LLM to generate high-level scenario understanding. The LLM output includes a rephrased scenario description, a scenario category (based on given scenario categories in the prompt), and an explanation for the classification. An embedding model will then calculate the embedding vector for the rephrased scenario description. Both the rule-based tags and the LLM-enhanced understanding are stored in a scenario database. During the scenario retrieval process, we employ a dual-path retrieval strategy, supporting both attribute-based search and similarity-based search. In the following subsections, we provide detailed descriptions of each submodule. We assume that the input BUS data has been preprocessed and includes stable track Unique Identifier (UUID).

A. Data Preprocessing

The input of each data log includes the ego vehicle's odometry information (x , y coordinates and yaw angle), and the perception output of detected objects, which includes their x , y , yaw angle, width, height, object category, and a tracking UUID.

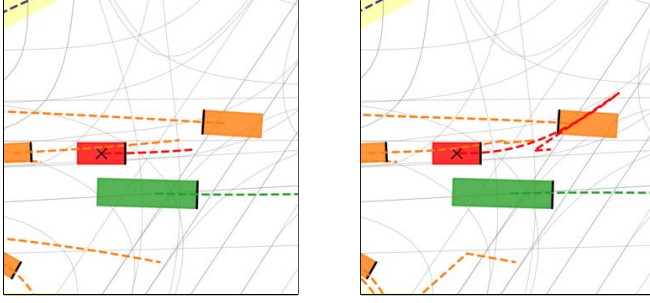
Although the input data are assumed to be refined before entering the system, they may still contain noise or missing entries. To mitigate these issues, we first apply linear interpolation within the valid lifespan of each object to fill in missing values. Then, a Savitzky-Golay smoothing algorithm [16] is employed to denoise the data while preserving sharp motion transitions. We specifically choose this method because other commonly used smoothing techniques, such as spline fitting, tend to over-smooth the data, suppressing short but critical events such as sudden braking, which may lead to the omission of important scenario patterns.

B. Rule-based Scenario Tagging

Following the work of [3], each scenario log is annotated with two types of tags: actor activities and actor–interaction activities. A key distinction from prior work is that our system does not rely on HD maps. This design choice is made to enhance the system's applicability in real-world scenarios where HD maps are often unavailable or unreliable. A detailed definition of each activity tag and the associated values is provided in Table I.

For the specific tagging rules related to "LongitudeTag", "LatitudeTag", "CollisionTag", "HeadingTag", "PositionTag", and "DistanceTag", we refer readers to the original work [3], where these rules are thoroughly described. To avoid redundancy, we do not repeat those details here.

In addition to the existing tags, we introduce a new actor–interaction tag called *TrajOverlapTag*. Unlike the *CollisionTag* in [3], which relies on motion prediction using a



(a) Predicted trajectory in 3 seconds based on Constant Turn Rate and Velocity (CTRV) model. No collision risk detected for *CollisionTag*

(b) Future actual trajectory in 3 seconds. High overlap risk detected for *TrajOverlapTag*

Fig. 3: Comparison between *CollisionTag* (left) with *TrajOverlapTag* (right). Red: ego vehicle; orange: guest vehicle; dash line: predicted / actual future trajectory.

CTRV model and Time to Collision (TTC) estimation, our method directly leverages the ground-truth future trajectories recorded in the logs. For both the ego vehicle and a surrounding object, we aggregate all 2D bounding boxes over a future time interval to form spatiotemporal motion envelopes. An overlap is determined based on whether these aggregated regions intersect. Depending on the time window in which overlap is detected, we assign three levels of severity: "HIGH" risk of overlap ($\Delta t = 1.5$ s), "LOW" risk of overlap ($\Delta t = 3.0$ s but not 1.5 s), or "NO" risk.

The formal definition is as follows. Let $\mathcal{B}_{\text{ego}}(t)$ and $\mathcal{B}_{\text{obj}}(t)$ denote the 2D bounding boxes of the ego vehicle and another object at time t . Define the aggregated motion regions:

$$\mathcal{A}_{\text{ego}}^{\Delta t} = \bigcup_{t \in [t_{\text{now}}, t_{\text{now}} + \Delta t]} \mathcal{B}_{\text{ego}}(t), \quad \mathcal{A}_{\text{obj}}^{\Delta t} = \bigcup_{t \in [t_{\text{now}}, t_{\text{now}} + \Delta t]} \mathcal{B}_{\text{obj}}(t)$$

An overlap indicator function is defined as:

$$\mathcal{O}(\Delta t) = \begin{cases} 1, & \text{if } \mathcal{A}_{\text{ego}}^{\Delta t} \cap \mathcal{A}_{\text{obj}}^{\Delta t} \neq \emptyset \\ 0, & \text{otherwise} \end{cases}$$

Then the *TrajOverlapTag* is assigned as:

$$\text{Tag} = \begin{cases} \text{HIGH}, & \text{if } \mathcal{O}(1.5 \text{ s}) = 1 \\ \text{LOW}, & \text{if } \mathcal{O}(3 \text{ s}) = 1 \text{ and } \mathcal{O}(1.5 \text{ s}) = 0 \\ \text{NO}, & \text{otherwise} \end{cases}$$

Compared to the *CollisionTag*, the proposed *TrajOverlapTag* better accounts for protective braking behaviors exhibited by the driver, even in the absence of an actual collision if the ego vehicle were to maintain its velocity. Fig. 3 shows an example where *TrajOverlapTag* correctly detects the guest vehicle triggering the ego vehicle's braking, while *CollisionTag* does not.

The output of rule-based scenario tagging is summarized in table II, where each scenario can be represented as several matrix with N agents and T timestamps.

TABLE II: Output structure after rule-based scenario tagging.

Tag Name	Shape	Description
TrackID	$[N, T]$	
ObjectCategory	$[N, T]$	
LongitudeTag	$[N+1, T]$	includes Ego vehicle (+1).
LatitudeTag	$[N+1, T]$	
HeadingTag	$[N, T]$	
PositionTag	$[N, T]$	
CollisionTag	$[N, T]$	
DistanceTag	$[N, T]$	
TrajOverlapTag	$[N, T]$	

C. Key Object Identification (KeyIdent)

In complex traffic scenarios, a large number of surrounding objects are typically detected within a single sequence, making it computationally exhaustive to perform LLM-based reasoning on every activity pair between the ego vehicle and each surrounding guest object. Furthermore, most surrounding objects are irrelevant to the ego vehicle's behavior. To address this, we introduce a KeyIdent module to filter only those candidates most likely to influence the ego vehicle's decisions.

We specifically focus on braking behavior as an interaction indicator. When the longitude status of the ego vehicle is either "DECELERATING" or "STANDING_STILL", and a surrounding object simultaneously exhibits either "LOW" or "HIGH" collision risk or "LOW" or "HIGH" trajectory overlap risk with the ego vehicle, the object is regarded as a key candidate responsible for the deceleration. This rule effectively filters out irrelevant objects and focuses LLM processing on those entities most likely to affect the ego vehicle's behavior.

D. LLM-based Scenario Understanding and Classification

1) *Scenario Description Composition*: In the scenario understanding module, a LLM is employed to interpret and categorize scenarios based on predefined scenario categories. As illustrated in Fig. 4a 4b, the process begins by transforming timestamp-based scenario tag sequences into a duration-based representation. Consecutive timestamps with identical activity tags are merged into a single segment, and a duration is assigned to each resulting segment.

Next, the duration-based segments are translated into a structured text description using a predefined template as shown in Fig. 4c. The generated description begins with an overview, including the total duration and the two primary actors involved in the scenario. This is followed by a detailed initialization of all activity tags. For subsequent segments, only the tags that change from the previous segment are included, along with their corresponding durations.

2) *LLM Prompt for Scenario Understanding*: The translated scenario description is combined with a predefined prompt and provided as input to the LLM, as shown in Fig. 4d. Following common prompt engineering principles [17], the prompt consists of several components: a role definition and instruction, a list of known scenario categories,

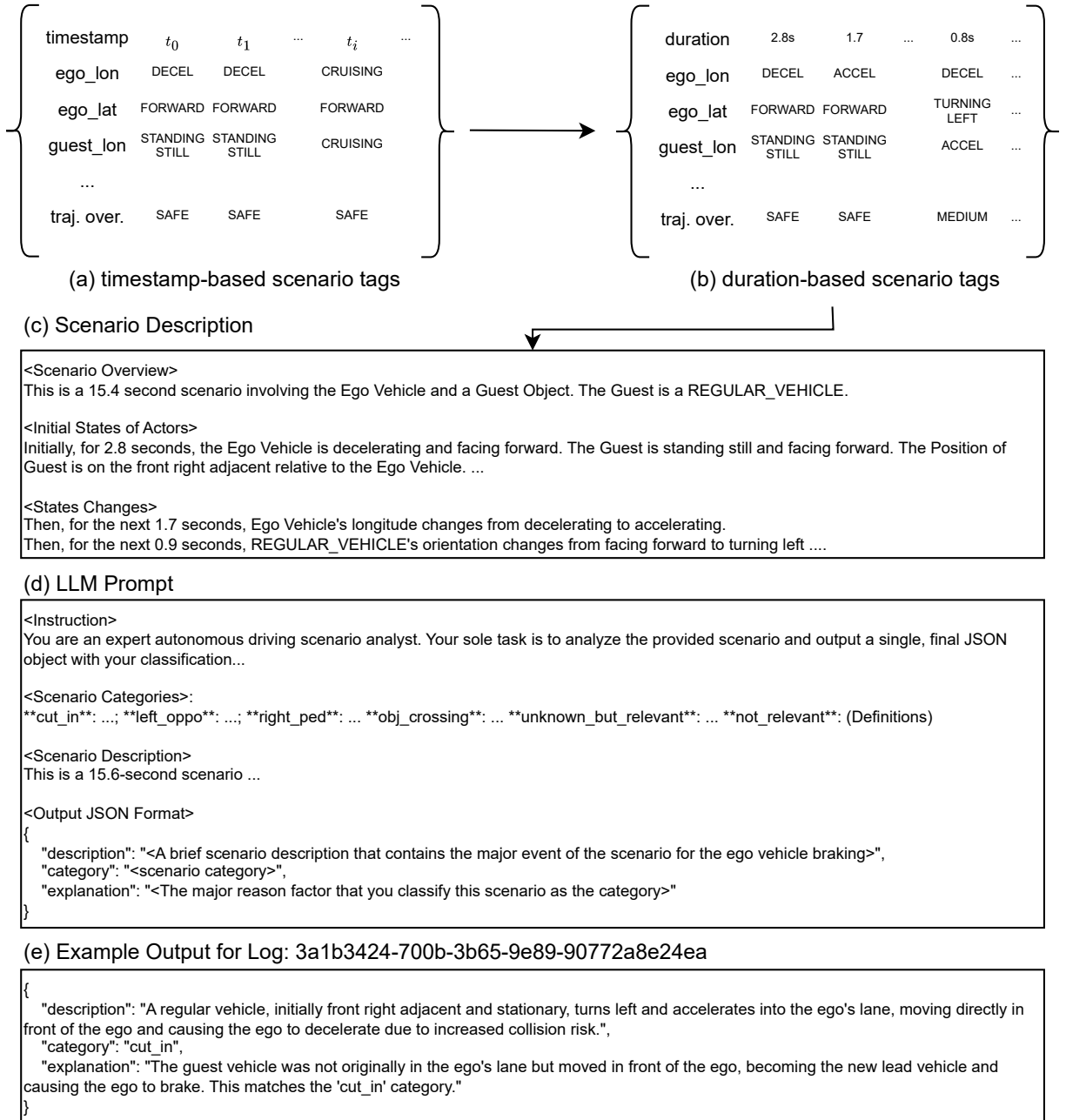


Fig. 4: Building scenario description and prompt to LLM and an output example

and an instruction to return the result in a fixed JSON format using specified keys.

Fig. 4e shows an output example, which includes the following fields:

- **Scenario Description:** A rephrased version of the rule-based scenario description that highlights the key event(s) in the scenario.
- **Scenario Category:** A selected category from the provided list of known scenario types that best fits the

current scenario. When none of them fit, output either **not_relevant** or **unknown_but_relevant**.

- **Explanation:** The reasoning behind the chosen category, explaining why the scenario was classified as such.

Finally, the rephrased scenario description is encoded using an embedding model to obtain a vector representation for similarity-based retrieval.

These four LLM-generated enrichments are stored alongside the rule-based tags in the scenario database for down-

TABLE III: Statistics of annotated scenario categories in Argoverse 2 Sensor Dataset train split (700 data logs).

Scenario Tag	Description	Count
cut_in	An object cuts into the path of the ego vehicle.	95
left_oppo	Ego vehicle turns left while an oncoming object approaches.	5
right_ped	Ego vehicle turns right while a pedestrian is crossing.	27
obj_cross	An object crosses in front of the ego vehicle.	64
ped_cross	A pedestrian crosses in front of the ego vehicle.	40
lead_brake	Sudden braking of the lead vehicle.	56
approach_stop	Ego vehicle approaches a stationary or slow-moving object.	91
not_relevant	Scenarios not relevant to ego vehicle behavior.	313
unknown_but_relevant	Hard-to-describe but relevant scenarios.	36
Total		727

stream retrieval and analysis.

E. Dual-Path Scenario Retrieval

Since only a closed set of scenario categories is explicitly provided in the prompt, OOD scenarios are not directly assigned by the LLM. To address this limitation, we propose a dual-path scenario retrieval module that supports both structured and open-ended queries.

For queries corresponding to known categories, the system performs direct retrieval by matching the `Scenario Category` field labeled by the LLM in the scenario database. In addition, users can retrieve scenarios using free-form scenario description queries. To enable this, we implement an embedding model to generate a vector representation for each scenario description. Cosine similarity is applied to compute similarity scores between the query and stored scenarios.

This dual-path retrieval framework allows users to access both standard scenario categories and customized scenarios described in natural language, enhancing the system's flexibility in handling both In-Distribution (ID) and OOD scenario retrieval.

IV. EXPERIMENTAL SETUP

This section outlines the experimental setup, beginning with dataset preparation by annotating scenario categories on Argoverse 2, followed by implementation details of the rule-based and LLM-based module.

A. Dataset

Our method is evaluated on the Argoverse 2 Sensor dataset [6], which consists of 1000 diverse urban driving logs featuring high-definition maps, time-synchronized multimodal sensor data (LiDAR, cameras, radar), and 3D object track annotations. We curate a set of high-level scenario category annotations focusing on challenging driving behaviors on the training data split (700 driving logs). As shown in table III, the scenarios are classified into nine categories, including seven specific scenario types as well as *not_relevant* and *unknown_but_relevant*. Each annotation consists of three elements: *log_id*, *guest_id*, and *scenario_category*. It should be noted that one data log may contain multiple scenarios, each involving different guest objects.

B. Implementation Details

We follow the rule-based scenario tagging setup from [3], with several refinements. The *LongitudeTag* remains unchanged. For *LatitudeTag*, we subdivide "TURNING" into "VEERING" and "TURNING": an accumulated turning angle greater than 0.17 radians is labeled as "TURNING," while an angle between 0.05 and 0.17 radians is labeled as "VEERING." For *CollisionTag*, TTC greater than 3 seconds is labeled "NO" risk, between 1.5 and 3 seconds as "LOW" risk, and less than 1.5 seconds as "HIGH" risk. For *PositionTag* and *HeadingTag*, angular thresholds are equally divided based on the number of defined classes. For *DistanceTag*, overlapping expanded bounding boxes are classified as "VERY_CLOSE"; distances below 10 meters as "CLOSE"; distances from 10 to 50 meters as "MEDIUM"; and distances over 50 meters as "FAR." Implementation details on *TrajOverlapTag* are provided in Section III-B.

For LLM-based modules, we evaluated three LLM models: GPT-4.1 [18], Gemini-2.5-Flash [19] and Qwen3-32B [20]. We primarily report the results of the best-performing model (gemini-2.5-flash-preview-05-20), along with a comparative analysis across all three models. For text embedding, we employ the opensource nomic embed model [21].

To ensure consistent and deterministic outputs from the LLM, we set the sampling parameters as follows: temperature = 0, top_p = 1.0, and top_k = 1. This configuration disables randomness in token sampling and makes the model generate the most likely output sequence given the prompt.

V. EVALUATION

We evaluate the effectiveness of our proposed method in comparison to the baseline rule-based tagging and retrieval approach introduced by [3].

Since our proposed framework is in two stages: KeyIdent followed by LLM-based scenario understanding, we organize the evaluation into four parts: (1) evaluation of the KeyIdent; (2) evaluation of scenario retrieval for known ID scenario categories; (3) evaluation of unknown scenario retrieval by query description; and (4) comparison of performance with different LLM models.

A. Evaluation for Key Object Identification (KeyIdent)

As shown in Table IV, the high recall of 0.86 ensures that most relevant pairs are captured. Although the precision is

TABLE IV: Evaluation for KeyIdent module

Category	Precision	Recall
cut_in	-	0.96
left_oppo	-	0.80
right_ped	-	0.74
obj_crossing	-	0.86
Overall	0.37	0.86

TABLE V: Evaluation for four known ID scenario categories.

Scenario Category	Precision	Recall	F1
	Baseline [3] / Ours		
cut_in	0.18 / 0.48	0.97 / 0.85	0.31 / 0.61
left_oppo	0.14 / 0.12	0.25 / 1	0.18 / 0.22
right_ped	0.60 / 0.58	0.375 / 0.63	0.46 / 0.60
obj_crossing	0.68 / 0.31	0.29 / 0.69	0.41 / 0.43
Overall	0.22 / 0.39	0.66 / 0.78	0.33 / 0.52

relatively low at 0.37, it is acceptable within our framework. The second-stage LLM module is specifically designed for scenario understanding and classification, allowing it to filter out false positives. Since the KeyIdent module does not differentiate between scenario categories, category-wise precision scores are not reported.

B. Evaluation for ID Scenarios

We first evaluate the scenario classification performance on four categories that are explicitly given in the prompt, as shown in Fig. 4d. To ensure a fair comparison, we carefully adjust the baseline retrieval filter condition to maximize its F1 score, making a trade-off between precision and recall.

As shown in Table V, our LLM-based method consistently outperforms the baseline in terms of F1 score across all categories as well as overall. While precision is lower in some categories, such as *obj_crossing*, it shows much stronger recall to ensure comprehensive scenario coverage. We notice the low precision on *left_oppo* and *obj_crossing* and analyze them in Section V-E.

C. Evaluation for OOD Scenario Description Retrieval

To evaluate the second retrieval path, which retrieval scenarios by a given OOD scenario description query, we evaluate three additional scenario categories that are not provided in the prompt: *approach_stop*, *lead_brake*, and *ped_crossing*. We report precision at top-10 (P@10) and recall at top-50 (R@50) based on similarity scores in Table VI.

The results show that similarity-based scenario retrieval generally performs lower than ID-based retrieval. However, given that it operates in a zero-shot setting, the performance remains acceptable. In particular, the recall scores for *lead_brake* and *ped_crossing* reach 0.5 and 0.67 at Top-50, respectively. The performance on *approach_stop* scenarios is slightly lower, likely because these scenarios always involve non-emergent braking, which may lead the LLM to underestimate their relevance during retrieval.

TABLE VI: Evaluation for Similarity-based Scenario Retrieval

Scenario Category	Query Description	P@10	R@50
approach_stop	"The ego vehicle approaches a stopped or slow-moving vehicle ahead, requiring careful deceleration to maintain a safe distance."	0.30	0.31
lead_brake	"The lead vehicle suddenly brakes, requiring the ego vehicle to brake in order to avoid a collision."	0.40	0.50
ped_crossing	"A pedestrian crosses in front of the ego vehicle, requiring the ego vehicle to brake or yield to avoid a collision."	0.30	0.67

TABLE VII: Comparison of method with different LLM models

Scenario Category	Precision	Recall	F1
Qwen3-32B	0.29	0.60	0.39
GPT-4.1	0.37	0.63	0.46
gemini-2.5-flash-preview-05-20	0.39	0.78	0.52

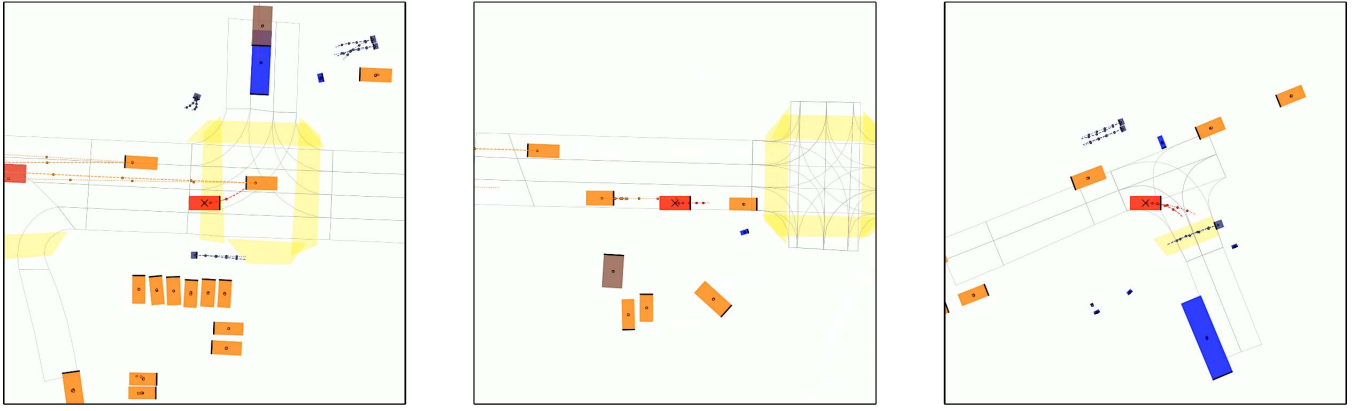
D. Comparison of LLM Models

We report the performance comparison of three LLM models: GPT-4.1, Gemini-2.5-Flash, and Qwen3-32B. The latter two are reasoning models, which require longer inference time. Among the three, Gemini-2.5-Flash achieves the best overall performance, with an F1 score of 0.52 and a recall of 0.78. In contrast, Qwen3-32B shows the lowest performance, primarily due to its limited token window, which is often exceeded by long scenario descriptions. All three models outperform the baseline method. This confirms the effectiveness of the proposed method.

E. Failure Case Analysis

Despite the promising results, we observe that certain braking events remain challenging to interpret correctly. In particular, the scenario category *left_oppo* shows the lowest performance among the four scenario types. Fig. 5a illustrates one example: while the ego vehicle is stopped in a waiting area for a left turn, an oncoming vehicle approaches and crosses in front of the ego vehicle. The absence of map information makes it difficult for the LLM to recognize intersections, causing confusion between *left_oppo* and *obj_crossing* scenarios, as both involve a vehicle crossing in front of the ego vehicle with similar patterns.

Another common failure case occurs during the KeyIdent, particularly in the scenarios *right_ped* and *approach_stop*. As illustrated in Fig. 5b and Fig. 5c, the ego vehicle applies only gentle, anticipatory braking rather than a hard brake, as the potential collision is foreseen early. Consequently, neither collision risk nor trajectory overlap is detected. This



(a) *left_oppo*: The ego vehicle starts braking before it turns left due to the oncoming vehicle. This scenario is mistakenly classified as *obj_crossing*."

(b) *approach_stop*: The ego vehicle approaches a stationary vehicle and initiates early braking. No collision risk detected.

(c) *right_ped*: The ego vehicle brakes early and stops in front of the crosswalk. No collision risk detected.

Fig. 5: Visualization of failure cases in scenario extraction. Red: ego vehicle; orange: guest vehicle; grey: pedestrian; dashed line: actual future trajectory; dotted line: predicted future trajectory.

suggests that the proposed method is more effective in handling critical braking scenarios, while it remains limited in capturing softer or anticipatory braking cases.

VI. CONCLUSION

In this work, we present a novel LLM-based scenario extraction and reasoning framework for identifying the causes of braking events in driving. By translating low-level BUS data into structured natural language descriptions, our method enables LLM-powered scenario understanding and classification. We introduce a dual-path retrieval method that supports both ID and OOD scenario retrieval. Furthermore, we provide scenario annotations on the Argoverse 2 sensor dataset to facilitate evaluation. Experimental results show that our approach not only outperforms baseline rule-based methods but also demonstrates strong zero-shot generalization to previously unseen OOD scenarios.

REFERENCES

- [1] B. Krzanich. (2016) Data is the new oil in the future of automated driving. Intel Newsroom.
- [2] E. De Gelder, J. Manders, C. Grappiolo, J.-P. Paardekoooper, O. O. Den Camp, and B. De Schutter, "Real-world scenario mining for the assessment of automated vehicles," in *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, 2020.
- [3] D. Guo, M. M. Sánchez, E. de Gelder, and T. P. van der Sande, "Scenario extraction from a large real-world dataset for the assessment of automated vehicles," in *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, 2023.
- [4] Y. Zhao, W. Xiao, T. Mihalj, J. Hu, and A. Eichberger, "Chat2scenario: Scenario extraction from dataset through utilization of large language model," in *2024 IEEE Intelligent Vehicles Symposium (IV)*, 2024.
- [5] C. Davidson, D. Ramanan, and N. Peri, "Refav: Towards planning-centric scenario mining," *arXiv preprint arXiv:2505.20981*, 2025.
- [6] B. Wilson, W. Qi, T. Agarwal, J. Lambert, J. Singh, S. Khandelwal, B. Pan, R. Kumar, A. Hartnett, J. K. Pontes *et al.*, "Argoverse 2: Next generation datasets for self-driving perception and forecasting," *arXiv preprint arXiv:2301.00493*, 2023.
- [7] N. Kalra and S. M. Paddock, "Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability?" *Transportation research part A: policy and practice*, 2016.
- [8] M. Scholtes, L. Westhofen, L. R. Turner, K. Lotto, M. Schuldes, H. Weber, N. Wagener, C. Neurohr, M. H. Bollmann, F. Körtke *et al.*, "6-layer model for a structured description and categorization of urban traffic and environment," *IEEE Access*, 2021.
- [9] D. Nalic, T. Mihalj, M. Bäuml, M. Lehmann, A. Eichberger, and S. Bernsteiner, "Scenario based testing of automated driving systems: A literature survey," in *FISITA web Congress*, 2020.
- [10] B. Schütt, J. Ransiek, T. Braun, and E. Sax, "1001 ways of scenario generation for testing of self-driving cars: A survey," in *2023 IEEE Intelligent Vehicles Symposium (IV)*, 2023.
- [11] D. Rempe, J. Phillion, L. J. Guibas, S. Fidler, and O. Litany, "Generating useful accident-prone driving scenarios via a learned traffic prior," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 17 305–17 315.
- [12] C. Sima, K. Renz, K. Chitta, L. Chen, H. Zhang, C. Xie, J. Beißwenger, P. Luo, A. Geiger, and H. Li, "Drivevm: Driving with graph visual question answering," in *European Conference on Computer Vision*, 2024.
- [13] X. Tian, J. Gu, B. Li, Y. Liu, Y. Wang, Z. Zhao, K. Zhan, P. Jia, X. Lang, and H. Zhao, "Drivevm: The convergence of autonomous driving and large vision-language models," *arXiv preprint arXiv:2402.12289*, 2024.
- [14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, 2022.
- [15] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM computing surveys*, 2023.
- [16] R. W. Schafer, "What is a savitzky-golay filter?[lecture notes]," *IEEE Signal processing magazine*, vol. 28, no. 4, pp. 111–117, 2011.
- [17] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.
- [18] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [19] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican *et al.*, "Gemini: a family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.
- [20] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
- [21] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic embed: Training a reproducible long context text embedder," *arXiv preprint arXiv:2402.01613*, 2024.